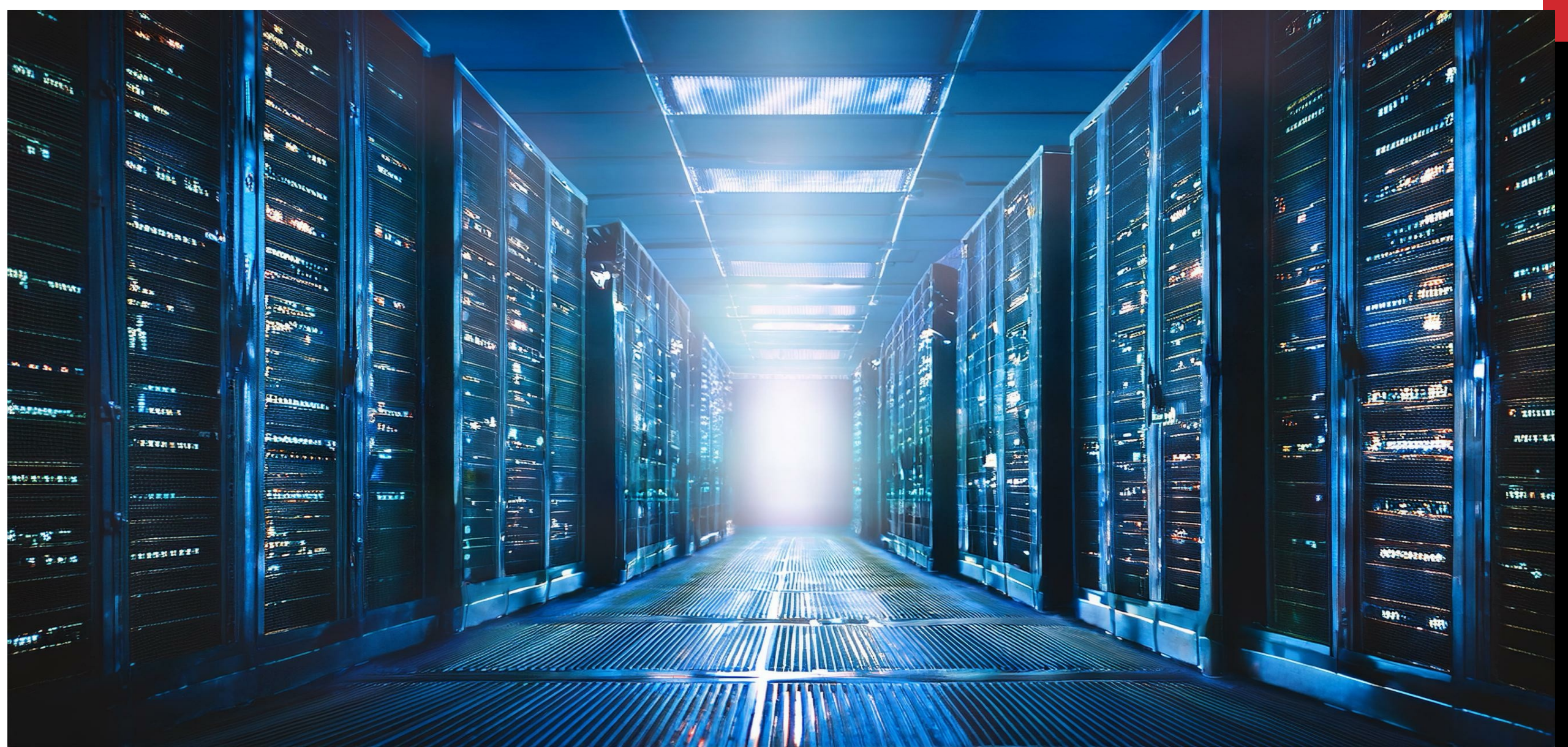


Monitoring



INTRODUCTION TO GRAFANA

Grafana is an open-source platform used to visualize, monitor, and explore data from many sources. It is a core tool for modern observability and incident response workflows.

- Web-based visualization platform
- Open source and widely adopted
- Used by SRE and platform teams
- Focused on observability use cases



WHAT GRAFANA IS

Grafana turns raw data into dashboards made of graphs, charts, and alerts. It helps teams understand system behavior at a glance.

- Dashboards and panels
- Time-series focused
- Query-based visualizations
- Supports alerting



<https://grafana.com/oss/grafana/>

WHY GRAFANA IS USED

Teams reach for Grafana because raw metrics and logs are hard to act on until someone can see them in context. An on-call engineer watching one dashboard can catch a memory leak climbing across nodes before it becomes a 2am outage.

- Application monitoring
- Infrastructure visibility
- Real-time dashboards
- Shared team views



COMMON GRAFANA USE CASES

The same Grafana instance serves very different jobs, from tracking response times to planning capacity months ahead. One team watches error rates during a release while another reviews the same data to justify next quarter's infrastructure spend.

- Response time tracking
- Error rate monitoring
- Capacity planning
- Incident investigation
- SLOs/SLAs
- Alerts



WHAT GRAFANA IS NOT

Grafana does not store metrics or logs by itself in most cases. It relies on external data sources to provide information.

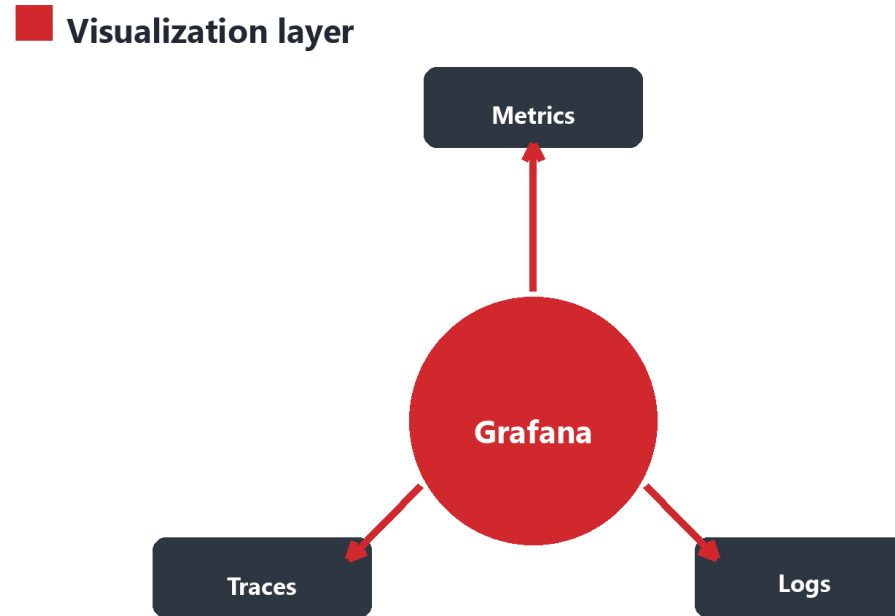
- Not a database
- Not a metrics collector
- Not a log storage engine
- Acts as a visualization layer



GRAFANA FOR OBSERVABILITY

Observability is about answering questions you did not plan for, and it starts with seeing your data. Grafana is the visual layer on top of your metrics, logs, and traces.

- Metrics visualization
- Log exploration
- Trace analysis
- Unified dashboards



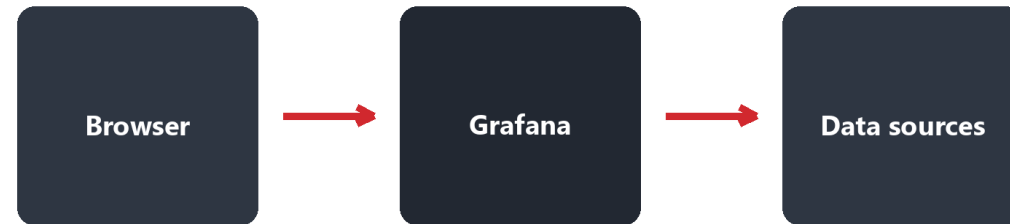
One view over your observability data

GRAFANA ARCHITECTURE BASICS

Grafana stays thin: it renders dashboards but stores almost nothing itself. It only queries external sources at view time, so a restart never loses data.

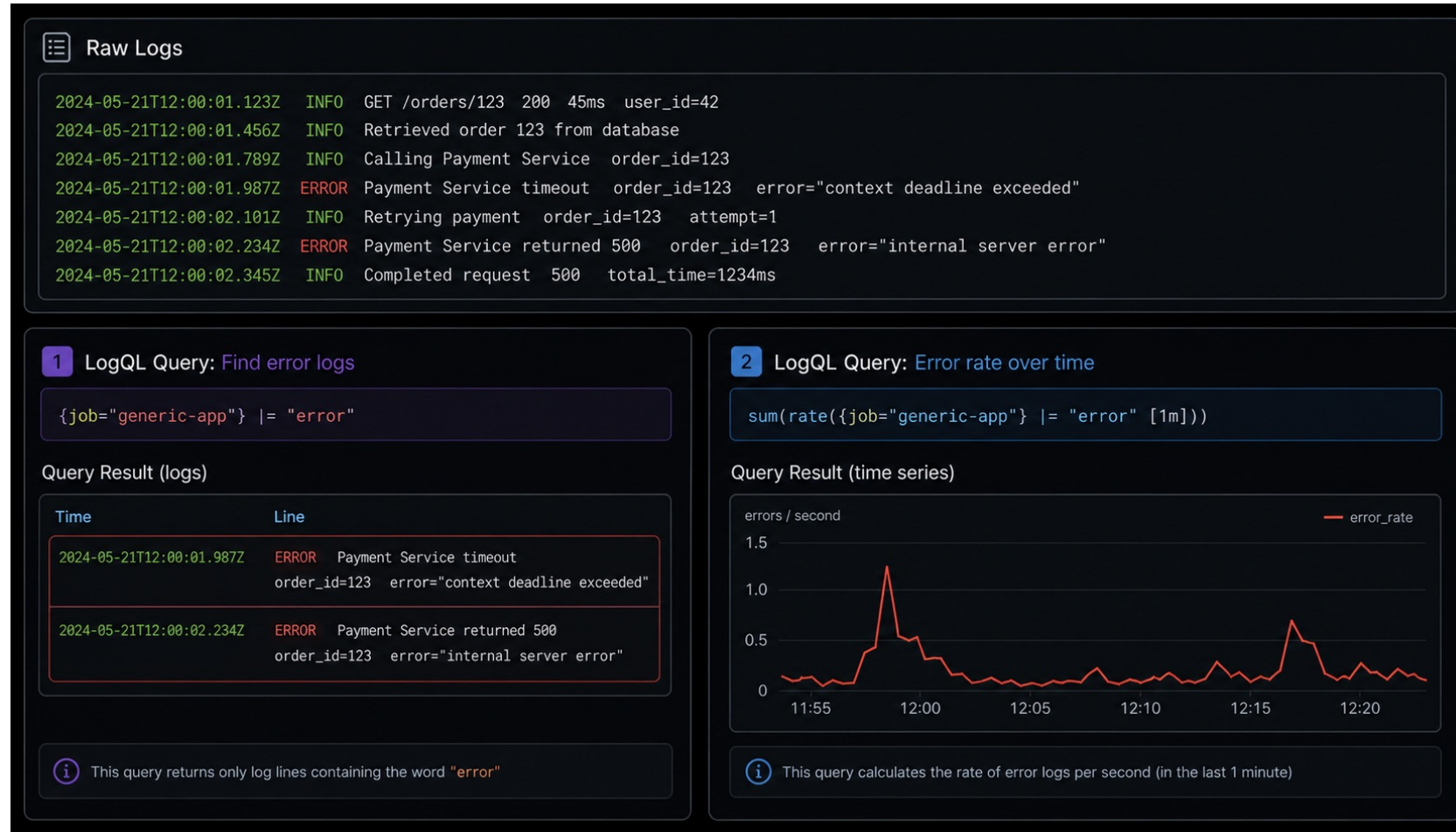
- Web-based UI
- Backend query engine
- External data sources
- Stateless by design

■ How Grafana runs



Queries at view time, stores nothing itself

LGTM STACK



The LGTM (Loki, Grafana, Tempo, Mimir) stack provides a 360 view of applications

Loki is a log aggregation system, often called Prometheus for logs. LogQL is used to write queries against the logs. Logs are useful for RCA (root cause analysis), troubleshooting, and detecting issues.



LGTM STACK

Raw Metrics (examples)

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{job="api",status="200",method="GET"} 10234
http_requests_total{job="api",status="500",method="GET"} 52
http_requests_total{job="api",status="200",method="POST"} 3121
http_requests_total{job="api",status="500",method="POST"} 18

# HELP http_request_duration_seconds HTTP request duration
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{job="api",le="0.1"} 4521
http_request_duration_seconds_bucket{job="api",le="0.25"} 8123
http_request_duration_seconds_bucket{job="api",le="0.5"} 11543
http_request_duration_seconds_bucket{job="api",le="1"} 14221
http_request_duration_seconds_bucket{job="api",le="2.5"} 15102
http_request_duration_seconds_bucket{job="api",le="5"} 15311
http_request_duration_seconds_bucket{job="api",le="+Inf"} 15320
http_request_duration_seconds_sum{job="api"} 32145.
http_request_duration_seconds_count{job="api"} 15320
```

Metric Key

Each line is a time series with:

- **Metric name** (e.g., `http_requests_total`)
- **Labels** (`key="value"`) that identify dimensions
- **Value** (counter, gauge, histogram, etc.)

1 PromQL Query: Availability (success rate)

```
100 * sum(rate(http_requests_total{job="api",status!="5.."}[5m]))
/ sum(rate(http_requests_total{job="api"}[5m]))
```

Query Result



Percentage of requests that did not return a 5xx status code (over the last 5 minutes).

2 PromQL Query: Latency (95th percentile)

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket{job="api"}[5m])) by (le))
```

Query Result

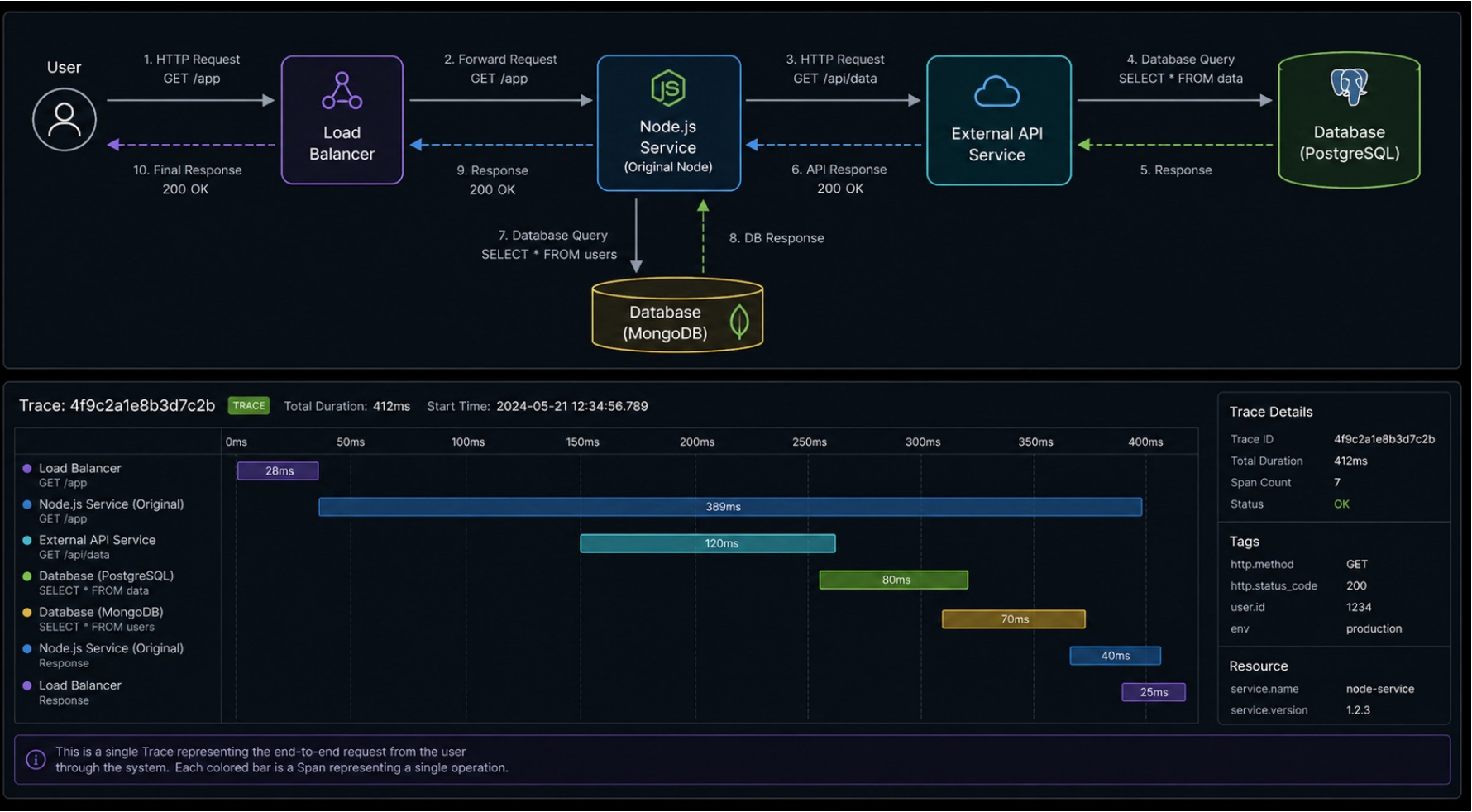


95th percentile of HTTP request duration (over the last 5 minutes).

Prometheus stores time-series metric data for monitoring systems. **Mimir** extends Prometheus into a scalable, distributed backend for long-term storage. Metrics, being structured and numeric, enable reliable tracking of SLOs, latency, and application performance.



LGTM STACK



Tempo stores distributed traces at scale. **Traces** provide end-to-end visibility into request flow, dependencies, and latency across services.

A trace captures one whole request from start to finish, and it breaks that request into units of work called spans.



RUNNING GRAFANA LOCALLY

Grafana can run locally so you can learn and break things without touching production. A single Docker container gives every student an identical setup in under a minute, and a restart resets it. The default login is admin / admin.

```
docker run -d -it -p 3000:3000 --name grafana grafana/grafana
```

Run Grafana locally

```
$ docker run -d -p 3000:3000 grafana/grafana  
  
open http://localhost:3000  
login admin / admin
```

One container, resets in seconds

GRAFANA NAVIGATION

Knowing where things live in Grafana saves real time during an incident, when seconds count. The left sidebar groups everything into dashboards, data connections, alerting, and admin, so you are not hunting through menus while a service is down.

- Dashboards section
- Connections menu
- Alerting features
- Administration settings

Grafana left sidebar

 Dashboards

 Explore

 Alerting

 Connections

 Administration

EXPLORING SAFELY

New users often hesitate to click around a production tool, worried they will break something. Grafana is built for exploration: querying and viewing data never changes the underlying source, so you can experiment freely while you learn.

- Read-only dashboards
- Preview queries
- Safe experimentation
- Undo and revert options

■ Safe by default

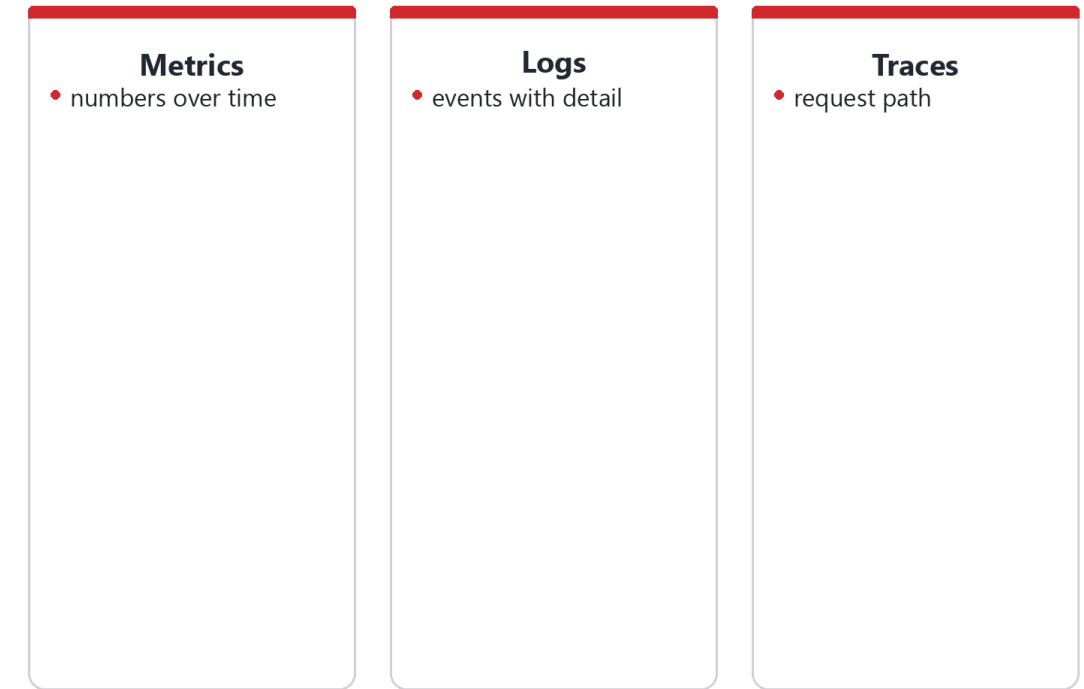
- ✓ View dashboards (read-only)
- ✓ Preview a query before running
- ✓ Experiment freely
- ✓ Data source never changes

OBSERVABILITY PILLARS OVERVIEW

Observability rests on three kinds of data, and each answers a question the others cannot. During an incident you lean on all three: metrics say something is wrong, logs say what happened, and traces say where.

- Metrics
- Logs
- Traces
- Used together, not in isolation

Three pillars



Used together, not in isolation

METRICS

Metrics are numeric measurements collected over time, and they are usually the first signal that something is off. When checkout latency drifts from 200ms toward 2s, the latency metric spikes long before a customer files a ticket.

- CPU, memory, disk usage
- Request rates and error rates
- Latency and throughput
- Efficient and low cost



LOGS

Logs are timestamped records of what an application actually did. When a payment fails, a metric tells you the error rate rose, but the log line shows the exact exception and the order ID behind it.

- Application logs
- System and service logs
- Structured or unstructured text
- High detail, higher volume



TRACES

A trace follows one request as it hops across services, recording how long each step took. When a single API call is slow but every service looks healthy on its own, the trace reveals the one downstream call eating 800ms.

- End-to-end request visibility
- Service-to-service latency
- Dependency mapping
- Critical for microservices



WHY ALL THREE MATTER

Relying on a single signal leaves blind spots that slow you down at the worst possible moment. Metrics show that something is wrong, logs explain why, and traces show where, and correlating the three is what turns a long outage into a short one.

- Metrics show something is wrong
- Logs explain why
- Traces show where
- Correlation is key

■ Each answers a question

Metrics

- WHAT is wrong

Logs

- WHY it happened

Traces

- WHERE it happened

Correlation is what speeds the fix

MONITORING VS OBSERVABILITY

Monitoring answers questions you already knew to ask; observability helps with the failures nobody predicted. A dashboard catches the disk filling up, but observability is how you track down why latency tripled after a deploy that 'changed nothing'.

- Monitoring is threshold-based
- Observability is exploratory
- Dashboards vs investigations
- Both are required



MTTD AND MTTR

Reliability is measured by two clocks: how fast you notice a problem and how fast you fix it. Cutting mean time to detect from 30 minutes to 2, and shrinking mean time to resolve, is the difference between a blip and a headline.

- MTTD: Mean Time To Detect
- MTTR: Mean Time To Resolve
- Faster detection saves downtime
- Context speeds resolution



WHY MONITORING MATTERS

If something runs in production and can fail, it needs to be monitored, because you cannot improve what you cannot measure. The first sign of a failing dependency is usually a metric, not an angry customer.

- Applications
- Infrastructure
- Networks
- Dependencies



SLOS AND SLAS

Reliability targets only mean something if you can prove whether you hit them. An SLO of 99.9 percent availability is just a slogan until monitoring data shows you spent 41 of your allowed 43 minutes of monthly downtime.

- SLOs define internal targets
- SLAs define customer promises
- Metrics validate compliance
- No data means no guarantees



GRAFANA AND OBSERVABILITY

During an incident, switching between five tools wastes the minutes that matter most. Grafana pulls metrics, logs, and traces into one view, so the same dashboard that flags a spike also lets you drill into the failing request.

- Single-pane-of-glass
- Cross-linked data
- Incident-focused dashboards
- Operational visibility



MCQ

Which observability pillar is best for detecting abnormal behavior quickly?

- Logs
- Metrics
- Traces
- Dashboards



ANSWER

Which observability pillar is best for detecting abnormal behavior quickly?

- Logs
- **Metrics**
- Traces
- Dashboards



MCQ

What observability pillar helps explain why an error occurred?

- Metrics
- Logs
- Traces
- Alerts



ANSWER

What observability pillar helps explain why an error occurred?

- Metrics
- **Logs**
- Traces
- Alerts



MCQ

What data type shows how a request flows across multiple services?

- Metrics
- Logs
- Traces
- Dashboards



ANSWER

What data type shows how a request flows across multiple services?

- Metrics
- Logs
- **Traces**
- Dashboards



MCQ

Why is monitoring required to track SLOs and SLAs?

- It stores application data
- It provides measurable reliability data
- It replaces incident response
- It eliminates downtime



ANSWER

Why is monitoring required to track SLOs and SLAs?

- It stores application data
- **It provides measurable reliability data**
- It replaces incident response
- It eliminates downtime

TELEMETRY FUNDAMENTALS

Telemetry is the data an application emits about its own behavior, before any tool collects it. A service that exposes its request counts and errors can be observed by any backend, which is why telemetry, not the dashboard, is the real foundation.

- Emitted by the application itself
- Collected by monitoring tools
- Independent of vendors
- Core to observability



TELEMETRY VS MONITORING TOOLS

It is easy to confuse telemetry with the tools that consume it, but telemetry exists first and stands on its own. Swap Prometheus for another backend and the application keeps emitting the same signals, because the telemetry belongs to the app.

- Application responsibility
- Tool agnostic
- Exists before dashboards
- Enables observability

Telemetry vs tools

Telemetry

- Emitted by the app
- Raw signal
- Tool agnostic
- Exists first

Monitoring tools

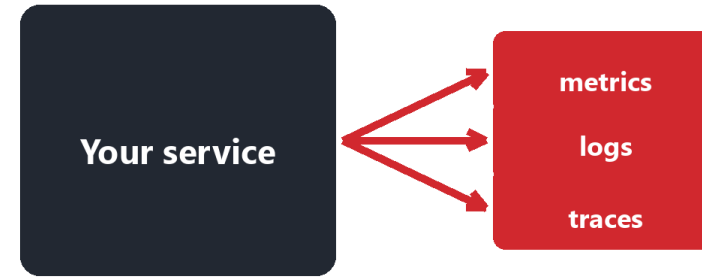
- Consume telemetry
- Store and visualize
- Swappable
- Built on top

WHY APPLICATIONS EMIT TELEMETRY

Without telemetry, a running service is a black box you can only guess about. Emitting metrics, logs, and traces turns that box transparent, so you diagnose a slow endpoint from data instead of redeploying with extra print statements.

- External visibility
- Reduced guesswork
- Inspectable behavior
- Operational transparency

■ Make the black box visible



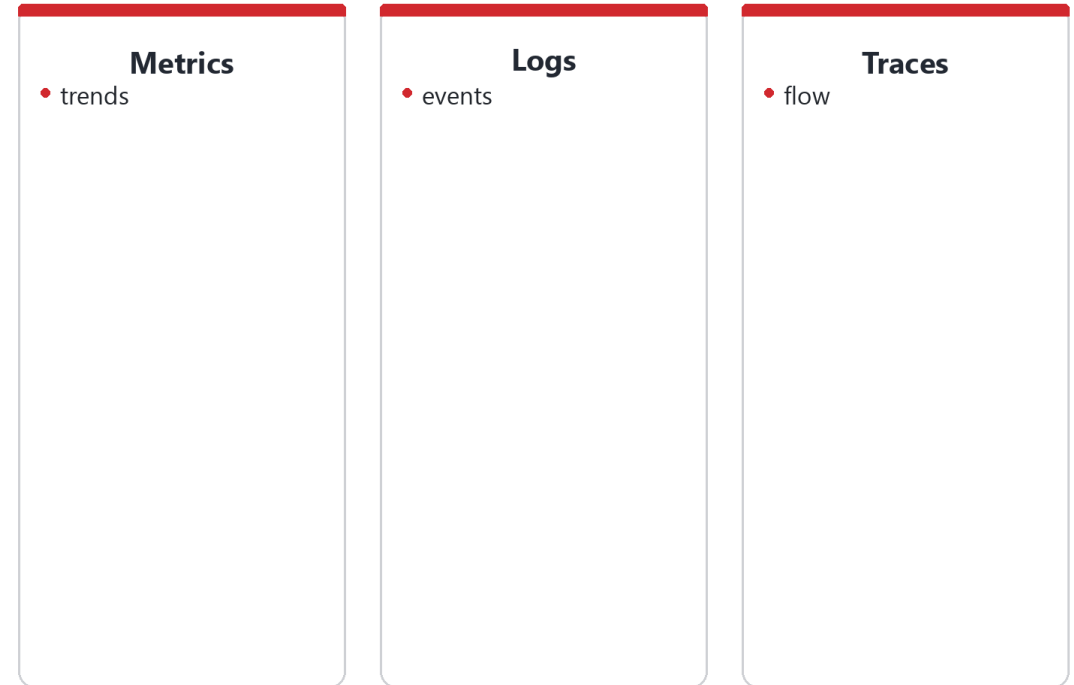
No telemetry means guesswork

TELEMETRY DATA TYPES

Telemetry usually arrives as metrics, logs, and traces, and each answers a different question. A latency metric flags the slowdown, a log explains the error, and a trace shows which service caused it.

- Metrics show trends
- Logs show events
- Traces show flow
- Used together

Telemetry data types

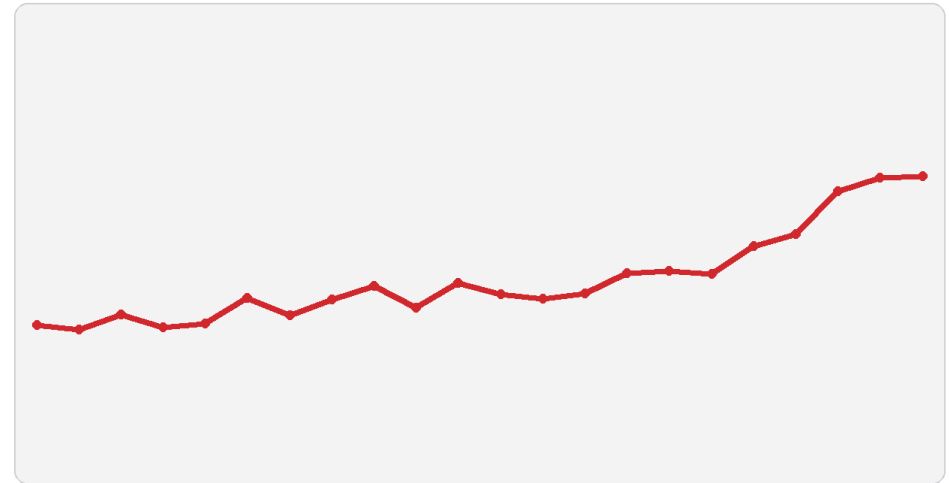


METRICS AS TELEMETRY

Metrics are the lightweight, always-on signal an application emits. Because a counter or gauge costs almost nothing to record, you can track request rates and errors at high frequency without slowing the service down.

- Request counts
- Error rates
- Latency
- Resource usage

■ Metrics as telemetry



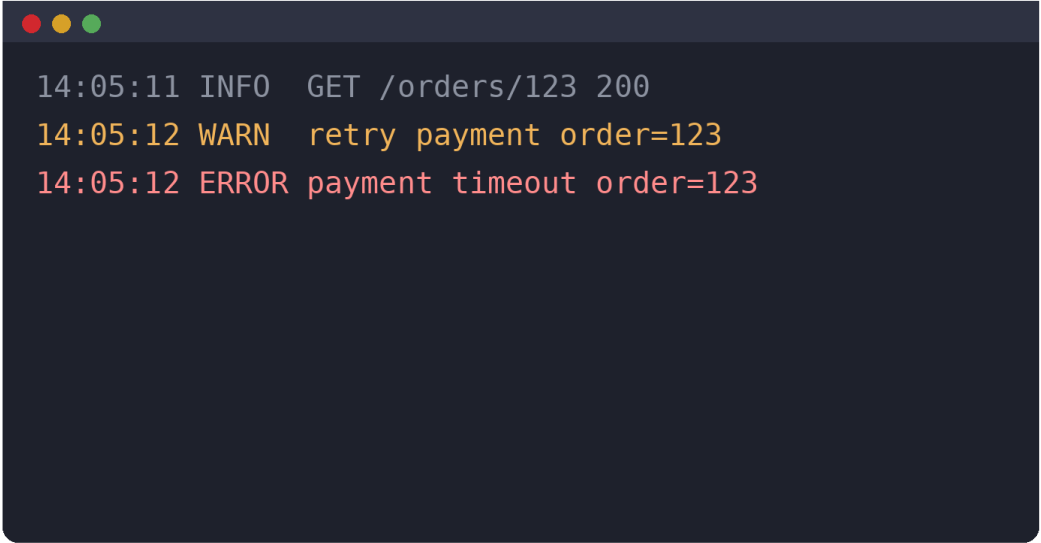
Cheap, continuous, high-frequency

LOGS AS TELEMETRY

Logs are the detailed story an application writes as it runs. They cost more to store than metrics, but when an order silently fails, the log line with the stack trace and order ID is what rescues the investigation.

- Event history
- High detail
- Debugging focused
- Human readable

■ Logs as telemetry

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It displays three log lines: a successful GET request, a warning about a payment retry, and an error about a payment timeout.

```
14:05:11 INFO GET /orders/123 200
14:05:12 WARN retry payment order=123
14:05:12 ERROR payment timeout order=123
```

Detail that explains a failure

TRACES AS TELEMETRY

In a distributed system, no single service has the whole picture of a request. A trace stitches the hops together, so when one checkout call is slow you can see it spent its time waiting on inventory, not the database.

- Request lifecycle
- Service boundaries
- Latency attribution
- Dependency insight

■ A trace and its spans

GET /checkout

auth

inventory

db query



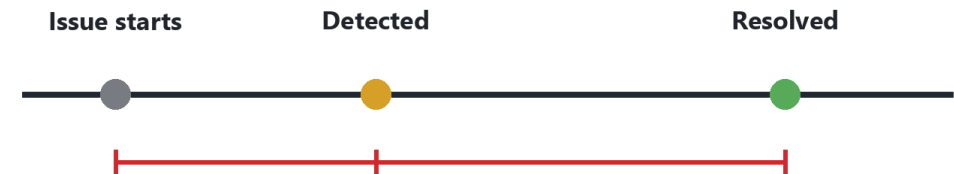
One request across many services

TELEMETRY IMPROVES MTTR AND MTTD

Good telemetry shortens both how fast you notice an issue and how fast you resolve it. When the data already points at the failing component, on-call spends minutes confirming a fix instead of hours forming theories.

- Lower MTTD
- Lower MTTR
- Faster root cause
- Reduced downtime

■ Telemetry shrinks both clocks



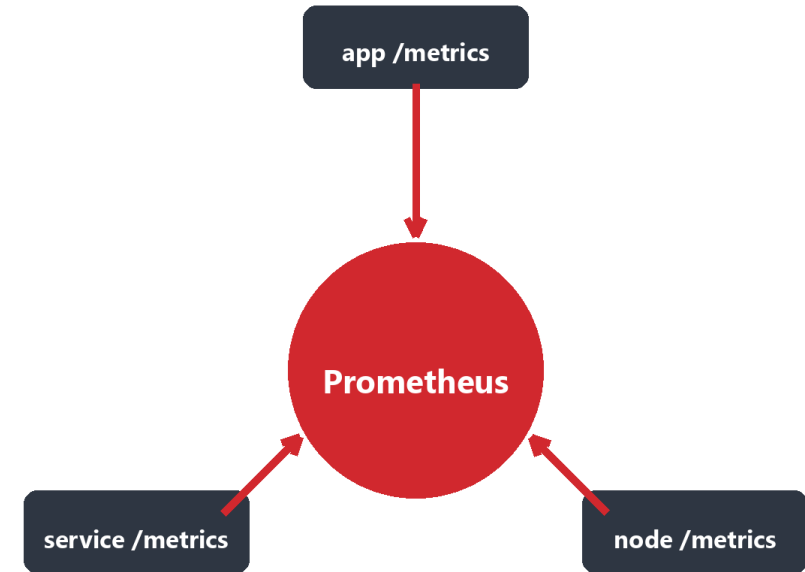
Lower MTTD and MTTR

METRICS ENDPOINTS

Most services expose a `/metrics` endpoint that publishes their telemetry in a standard text format. A monitoring system scrapes that endpoint on a schedule, so adding observability can be as simple as exposing one HTTP path.

- `/metrics` endpoint
- Machine readable
- Pull based collection
- Time series data

■ Metrics endpoints



Scrapes `/metrics` on a schedule

TELEMETRY AND RELIABILITY TARGETS

You cannot prove an SLO without the data behind it, and that data is telemetry. Failed-request and latency signals are what let you say with confidence that you stayed inside your error budget this month.

- Failed request rates
- Latency thresholds
- Availability tracking
- Reliability evidence

■ Reliability targets need data



43 min/month error budget

MCQ

What best describes telemetry?

- A monitoring dashboard
- An application emitting data about itself
- A logging format
- A metrics database



ANSWER

What best describes telemetry?

- A monitoring dashboard
- **An application emitting data about itself**
- A logging format
- A metrics database



MCQ

Which telemetry type is primarily used to calculate SLOs?

- Logs
- Traces
- Metrics
- Alerts



ANSWER

Which telemetry type is primarily used to calculate SLOs?

- Logs
- Traces
- **Metrics**
- Alerts



MCQ

Why does telemetry reduce MTTR?

- It replaces incident response
- It provides context and visibility
- It prevents failures entirely
- It scales infrastructure automatically



ANSWER

Why does telemetry reduce MTTR?

- It replaces incident response
- **It provides context and visibility**
- It prevents failures entirely
- It scales infrastructure automatically



MCQ

What is the purpose of the /metrics endpoint?

- To expose logs
- To store metrics
- To expose telemetry for scraping
- To visualize dashboards



ANSWER

What is the purpose of the /metrics endpoint?

- To expose logs
- To store metrics
- **To expose telemetry for scraping**
- To visualize dashboards



NEXT LAB: CENTRALIZED LOGGING WITH LOKI

In this lab you will ship the logs your API emits into one searchable place. Instead of opening a shell on each container, you will query every service's logs from a single screen, the way on-call engineers work during an incident.

- Centralized logging
- Application visibility
- Incident investigation
- Log correlation



WHY CENTRALIZED LOGGING

Logs scattered across dozens of containers are nearly useless when you are under pressure. Centralizing them means one search can follow a failing request across every service, with no SSH and no guessing which host handled it.

- Single source of truth
- Searchable history
- Cross-service visibility
- Faster debugging

Why centralize

Scattered logs

- One host at a time
- SSH to each box
- Lost on restart

Centralized

- One searchable store
- Follow a request anywhere
- Retained history

CENTRALIZED LOGGING OVERVIEW

Centralized logging funnels logs from many services into a single store built for search and retention. When an incident spans three services, you read one correlated timeline instead of stitching together separate log files.

- Single log location
- Searchable history
- Multi-service visibility
- Operational at scale



WHY DOCKER LOGS IS NOT ENOUGH

docker logs is fine for one container on your laptop, but it falls apart in production. It shows a single container, keeps no searchable history, and vanishes when the container restarts, which is exactly when you needed the logs.

- One container at a time
- No historical search
- No correlation
- Not team friendly

■ docker logs is not enough

docker logs

- One container
- No history
- Gone on restart

Centralized logging

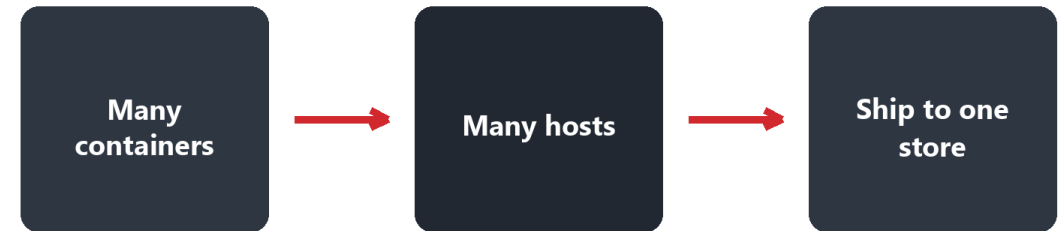
- Every service
- Searchable history
- Survives restarts

LOGGING AT SCALE PROBLEM

Real systems run many short-lived containers across many hosts, and any of them can hold the clue you need. When a container crashes and is rescheduled, its local logs disappear, so logs must be shipped somewhere central to survive.

- Many containers
- Frequent restarts
- Distributed systems
- Shared responsibility

■ Logging at scale



Local logs vanish when pods restart

WHAT IS LOKI

Loki is a log aggregation system from Grafana Labs, often called 'Prometheus for logs.' It is built to store large volumes of logs cheaply by indexing a few labels instead of the full text of every line.

- Log aggregation
- Label-based indexing
- Grafana integration
- Cost efficient

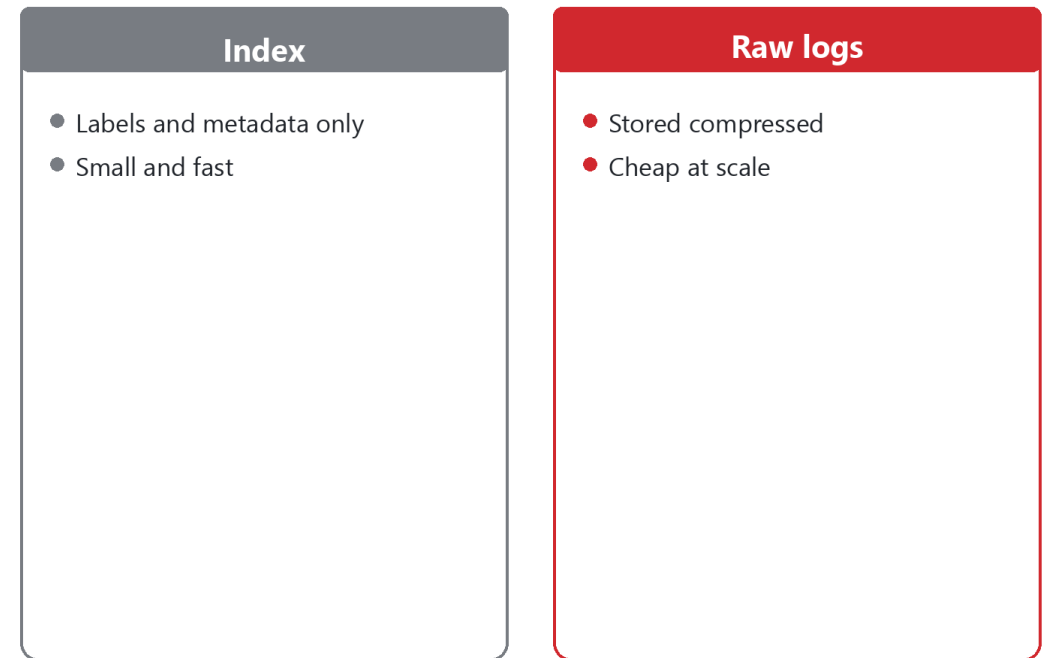


LOKI DESIGN PHILOSOPHY

Traditional log systems index every word, which is powerful but expensive at scale. Loki instead indexes only a small set of labels and keeps the raw logs compressed, trading heavyweight search for much lower cost and simpler operations.

- Labels not full text
- Low storage cost
- Fast queries
- Observability focused

■ Loki design philosophy



THE LOGGING STACK

Centralized logging is not one product but a small pipeline of cooperating parts. An application writes logs, a shipper collects them, a store like Loki holds them, and Grafana is where you finally read them.

- Application
- Log shipper
- Log store
- Visualization layer

■ The logging stack



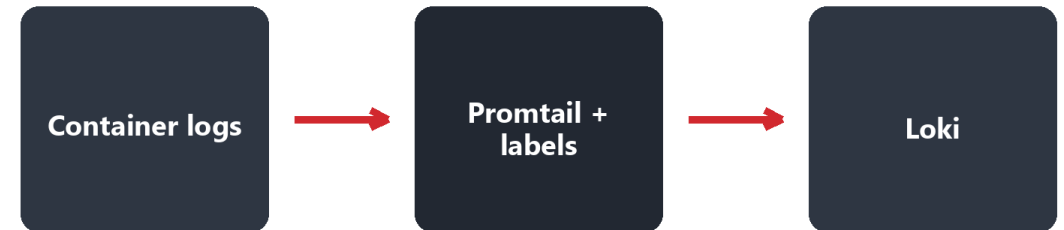
Each part has one job

PROMTAIL ROLE

Logs only become useful once something carries them from the container to the store. Promtail does that job in this lab: it tails each container's output, adds labels like service and environment, and pushes to Loki. Grafana now recommends Alloy as its modern replacement.

- Reads container logs
- Adds labels
- Ships to Loki
- Lightweight agent

■ Promtail role



Grafana Alloy is the modern replacement

GRAFANA AND LOGS

Once logs live in Loki, Grafana becomes where the whole team reads them. Nobody needs SSH access or production credentials, because querying and filtering logs happens in the browser under the same access controls as every dashboard.

- Explore view
- Search and filter
- Time based queries
- Team accessible

■ Read logs in Grafana Explore

A screenshot of the Grafana Explore interface. It features a dark theme with a window-like header containing three colored dots (red, yellow, green). The text 'Data source: Loki' is displayed in orange. Below it, a log query '{container="telemetry-api"} |= "ERROR"' is shown in green.

```
Data source: Loki

{container="telemetry-api"} |= "ERROR"
```

In the browser, no SSH

FROM CLI TO CENTRALIZED LOGS

Ad hoc grep over SSH does not scale and cannot be shared. Centralized logging turns a one-off command into a saved, repeatable query that a teammate can rerun next week against the same incident window.

- No grep pipelines
- Saved queries
- Shared dashboards
- Audit friendly

■ From CLI to centralized

grep over SSH

- One-off command
- Per host
- Not shareable

Saved LogQL

- Repeatable
- Shared with the team
- Audit friendly

WHEN CENTRALIZED LOGGING MATTERS

Centralized logging earns its keep the first time you hit a real production incident. With services multiplying and an on-call rotation handing off at 2am, a shared, searchable log history is what lets the next person pick up where the last left off.

- Production incidents
- Multiple services
- On-call rotations
- Post-incident reviews

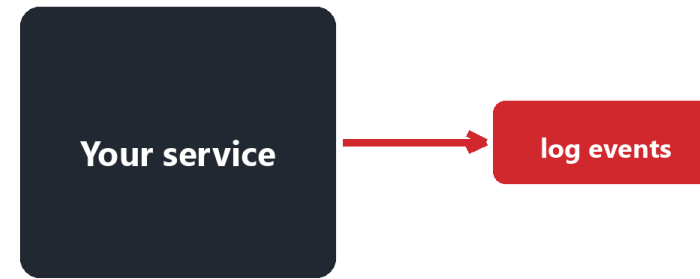


LOGS AS TELEMETRY SIGNALS

Logs are not separate from observability; they are one of its core telemetry signals. Each line is an event the application chose to record, which is why one good log message is worth more than a dozen vague ones.

- Application emitted
- Event driven
- Human readable
- Operational signal

■ Logs are a telemetry signal



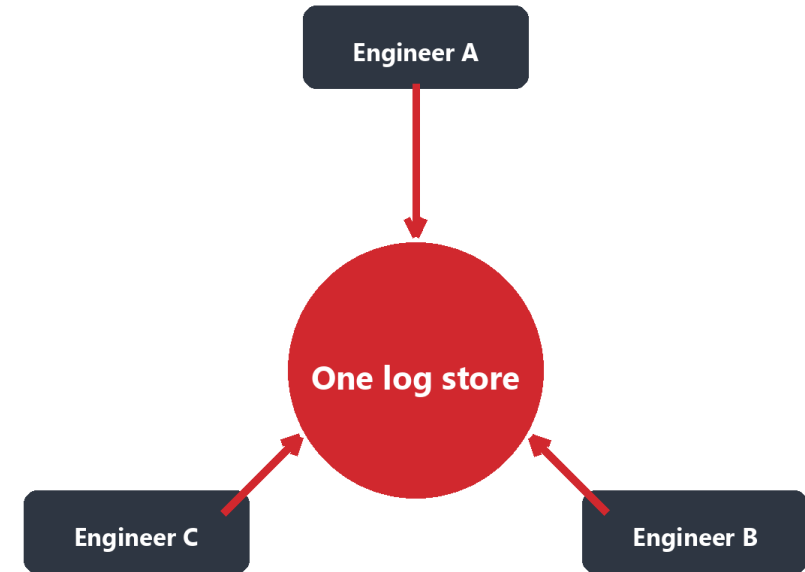
Each line is an event the app recorded

WHY CENTRALIZE LOGS

A single source of truth for logs changes how a team investigates. Instead of three engineers grepping three hosts, everyone queries the same store and sees the same evidence, which speeds both the fix and the follow-up review.

- Search once
- Query across services
- Team access
- Historical retention

■ A single source of truth



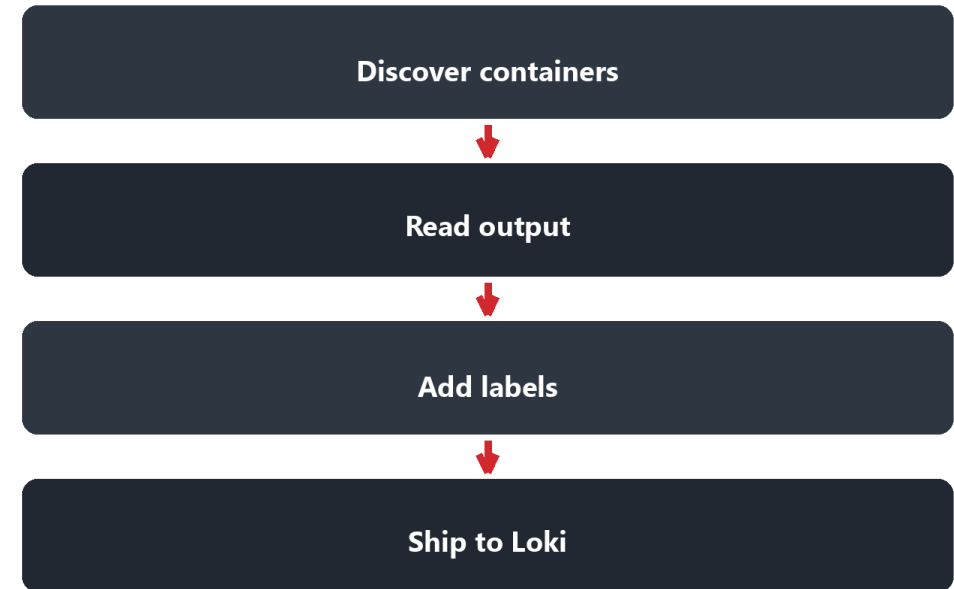
Everyone sees the same evidence

PROMTAIL LOG COLLECTION

Promtail is deliberately simple: it discovers running containers, reads their output, and forwards it with labels attached. Running close to the source lets it tag each stream with where it came from before Loki ever sees it.

- Reads container logs
- Discovers Docker containers
- Adds metadata
- Ships to Loki

■ Promtail log collection

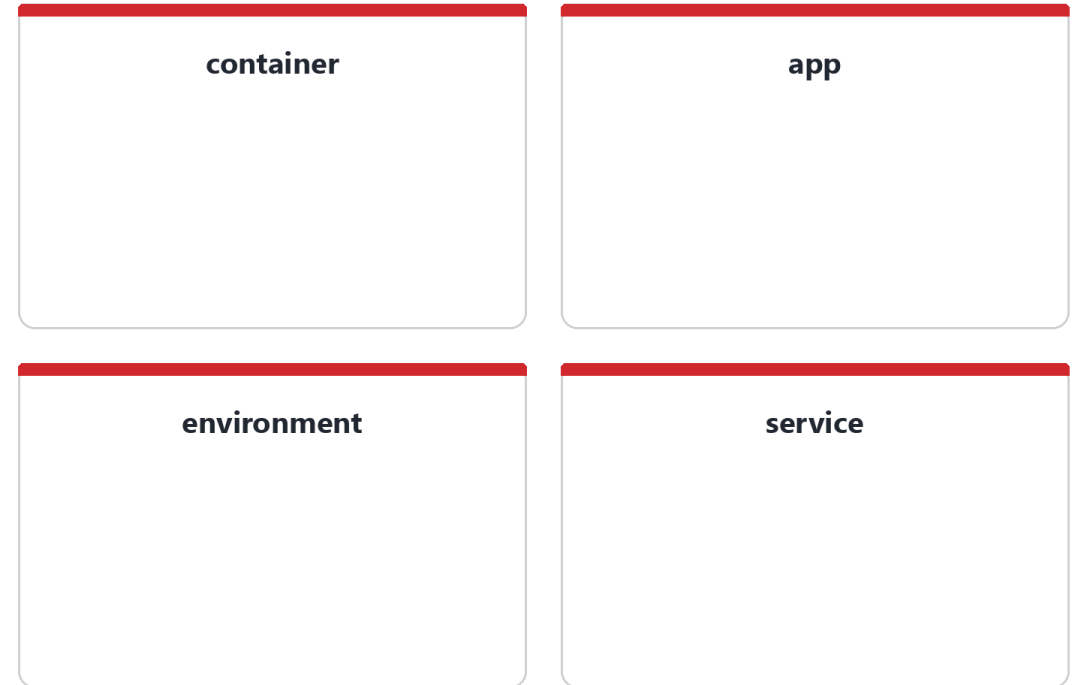


LABELS IN LOKI

Labels in Loki describe the stream a log belongs to, not the words inside it. Choosing labels like service, environment, and container is what later lets you say 'show me only the payment service in production' in one expression.

- Container name
- Application name
- Environment
- Service identity

■ Labels describe the stream



Not the words inside the log

WHY LABELS MATTER

Labels are how Loki stays fast and cheap, by narrowing the data before it ever scans text. Querying one well-labeled stream reads a fraction of the data a blind full-text search would, which keeps queries quick during an incident.

- Pre-filtering
- Reduced scan scope
- Predictable queries
- Scalable design

■ Why labels matter



Narrow the data before scanning text

LOGQL BASICS

LogQL is Loki's query language, and it borrows ideas you already know from PromQL and grep. You select a stream by its labels, then filter the lines, so anyone comfortable with grep can be productive in minutes.

- Label selectors
- Line filters
- Regex support
- Time awareness

■ LogQL = labels + filter

```
{app="telemetry-api"}  
  select the stream by its labels  
  
|= "ERROR"  
  filter the lines
```

SELECTING LOG STREAMS

Every LogQL query starts by choosing which streams to read, using labels in curly braces. Writing `{container="telemetry-api"}` narrows millions of lines down to one service before any text matching happens.

- `{container="telemetry-api"}`
- Label based selection
- Efficient querying
- Predictable results

■ Selecting log streams

```
{container="telemetry-api"}  
  
select stream by labels first
```

FILTERING LOG LINES

After selecting a stream, you filter for the lines that matter, much like piping to `grep`. `|=` matches text, `|~` matches a regex, and `!=` excludes noise, and you can chain them to zero in on a failure.

- `|=` contains
- `|~` regex match
- `!=` exclude
- Composable filters

Filtering log lines

```
|= "text"      contains
|~ "re.*x"     regex match
!= "text"      does not contain
```

FROM GREP TO LOGQL

If you already think in grep, LogQL will feel familiar. `grep ERROR` becomes `|= "ERROR"` and `grep -v DEBUG` becomes `!= "DEBUG"`, so your muscle memory carries over almost directly.

- `grep ERROR`
- `|= "ERROR"`
- `grep -v DEBUG`
- `!= "DEBUG"`

■ From grep to LogQL

grep	LogQL
<code>grep ERROR</code>	<code> = "ERROR"</code>
<code>grep -v DEBUG</code>	<code>!= "DEBUG"</code>
<code>grep -E "re"</code>	<code> ~ "re"</code>

LOGS DURING INCIDENTS

Metrics tell you something broke; logs tell you the story of how. When error rate spikes, the matching log lines carry the exception, the stack trace, and the IDs you need to reproduce and fix the issue.

- Error messages
- Warnings
- Execution flow
- Root cause clues

■ Logs during incidents



Metrics detect, logs explain

USING GRAFANA EXPLORE FOR LOGS

Grafana Explore is the workbench for ad hoc log investigation, separate from saved dashboards. During an incident you pick Loki, write a quick LogQL query, and live-tail the results as new logs arrive.

- Explore view
- Time range selection
- Live tailing
- Ad hoc analysis



SELECTING A DATA SOURCE

Grafana can talk to many backends, so every query needs to know where to go. Choosing Loki as the data source in Explore is what sends your LogQL to the log store rather than to a metrics database.

- Loki data source
- Explore dropdown
- Query context
- Consistent results

■ Selecting a data source

- ✓ Open Explore
- ✓ Pick the Loki data source
- ✓ Write your LogQL
- ✓ Run and read results

BUILDING A LOGQL QUERY

Good LogQL is built in layers, narrowing the data at each step. Select the labels first to cut the scope, then add line filters, which keeps queries fast and the results readable.

- Select labels first
- Filter lines second
- Reduce noise
- Improve performance

■ Build a query in layers

```
{app="api"}  
  narrow the scope (labels)  
|= "ERROR"  
  then filter lines
```


TIME RANGE MATTERS

Every log query runs inside a time window, and the window size drives both speed and relevance. Narrowing to the ten minutes around an incident returns results faster and hides the noise from hours that do not matter.

- Incident windows
- Reduce scan size
- Faster feedback
- Accurate context

■ Time range matters

Incident window



Smaller window, faster and clearer

FINDING ERRORS QUICKLY

Errors are usually the first thing you reach for during an incident. Filtering a labeled stream for ERROR turns a flood of logs into the handful of lines that explain the failure, and you can save that query for next time.

- Filter ERROR
- Combine with labels
- Save queries
- Share findings

■ Finding errors quickly

```
{app="api"} |= "ERROR"
```

Usually the first clue

COMBINING MULTIPLE FILTERS

Real investigations rarely match on a single word. Chaining filters, such as keeping lines that contain ERROR and also mention a specific order ID, narrows thousands of lines down to the exact sequence you care about.

- AND logic
- Multiple |= operators
- Noise reduction
- Targeted results

■ Combining filters (AND)

```
{app="api"}  
  |= "ERROR"  
  |= "order-123"
```

EXCLUDING LOG NOISE

Healthy systems are noisy, and debug lines and health checks can bury the signal. Excluding them with `!=` lets the failures stand out, so you read ten relevant lines instead of ten thousand routine ones.

- Exclude DEBUG
- Exclude health checks
- Focus on failures
- Cleaner output

■ Excluding log noise

```
{app="api"}  
  != "DEBUG"  
  != "health"
```


SAVED LOG QUERIES

The query that cracked last week's incident should not be retyped from memory. Saving and sharing common LogQL queries turns one engineer's hard-won knowledge into a repeatable tool for the whole on-call team.

- Repeatable queries
- Team shared knowledge
- Operational consistency
- Faster response

■ Saved log queries

✓ Errors in the last 15m

✓ 5xx by service

✓ Payment failures

✓ Slow requests

LOGS AND METRICS TOGETHER

Metrics and logs are strongest read side by side. A latency graph tells you the slowdown started at 14:05, and jumping to the logs for that exact minute shows the timeout that caused it.

- Metrics detect
- Logs explain
- Complementary signals
- Correlated analysis

■ Logs and metrics together



The graph shows when, the logs show why

LOGS IN POST INCIDENT REVIEWS

After an incident, logs provide the factual timeline that keeps a review honest. Instead of arguing from memory, the team reconstructs exactly what happened and when, which is how the same outage gets prevented next time.

- Event timelines
- Root cause analysis
- Evidence based reviews
- Reliability improvement



MCQ

What problem does centralized logging primarily solve?

- Reducing application latency
- Searching and correlating logs across services
- Replacing metrics systems
- Eliminating application errors



ANSWER

What problem does centralized logging primarily solve?

- Reducing application latency
- **Searching and correlating logs across services**
- Replacing metrics systems
- Eliminating application errors



MCQ

What is Loki optimized to index?

- Full log message text
- Application binaries
- Log metadata and labels
- Database records



ANSWER

What is Loki optimized to index?

- Full log message text
- Application binaries
- **Log metadata and labels**
- Database records



MCQ

What is the main role of Promtail?

- Visualize logs
- Store logs long term
- Collect and ship logs to Loki
- Generate application logs



ANSWER

What is the main role of Promtail?

- Visualize logs
- Store logs long term
- **Collect and ship logs to Loki**
- Generate application logs



MCQ

What is the first step in a LogQL query?

- Filter log lines
- Select log streams using labels
- Apply regex matching
- Aggregate results



ANSWER

What is the first step in a LogQL query?

- Filter log lines
- **Select log streams using labels**
- Apply regex matching
- Aggregate results



MCQ

Why are labels important in Loki?

- They format log messages
- They allow efficient pre-filtering
- They replace log content
- They store metrics



ANSWER

Why are labels important in Loki?

- They format log messages
- **They allow efficient pre-filtering**
- They replace log content
- They store metrics



KUBERNETES MONITORING ENVIRONMENT

This lab runs a complete observability stack inside Kubernetes, deployed and managed the way a real platform team would. Everything is wired together up front, so you can focus on exploring a production-style system instead of installing one.

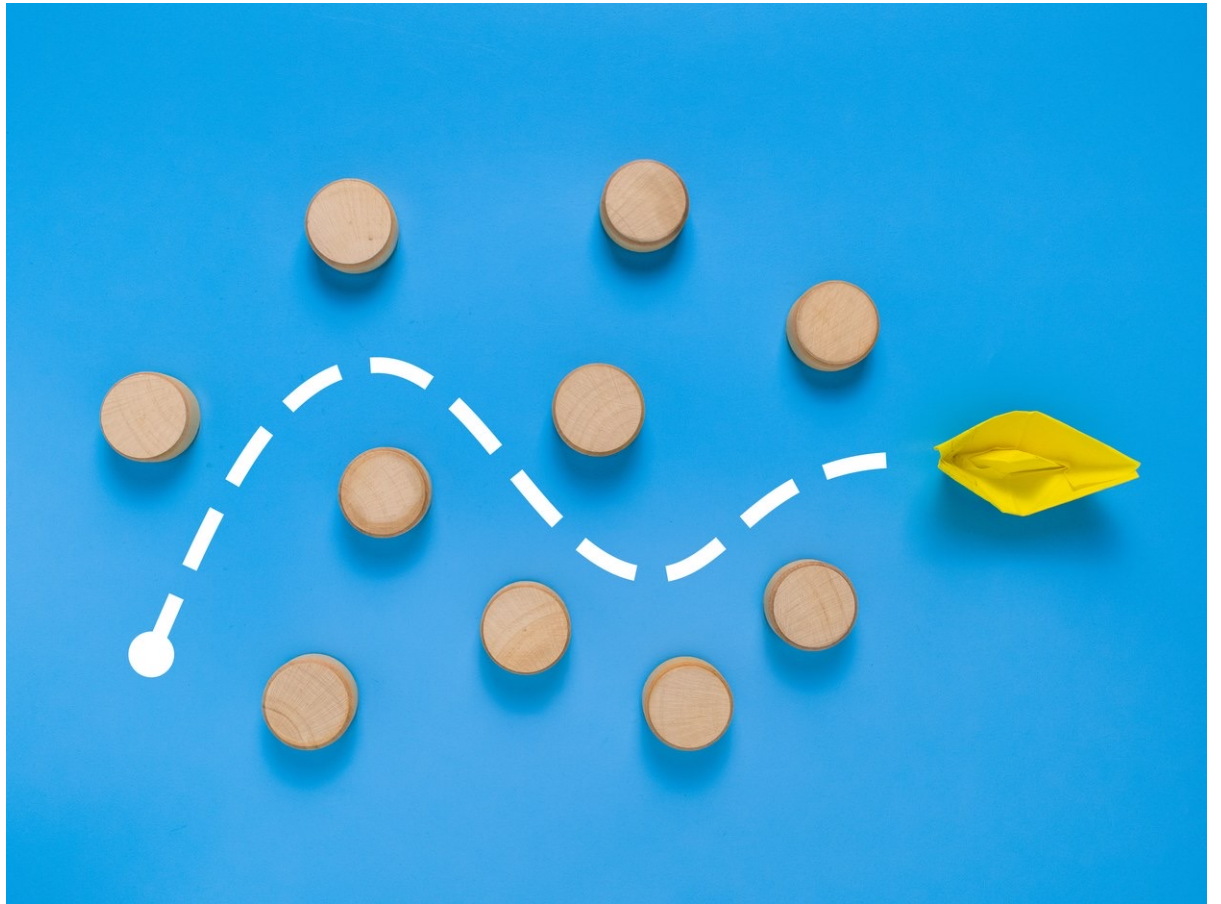
- Cloud native environment
- GitOps managed
- Production style setup
- End to end visibility



WHY MICROSERVICES NEED OBSERVABILITY

Splitting an application into many services buys flexibility but scatters the evidence when things break. A single user request might touch eight services, so understanding a failure means seeing across all of them, not just one.

- Many services
- Network boundaries
- Independent deployments
- Shared infrastructure

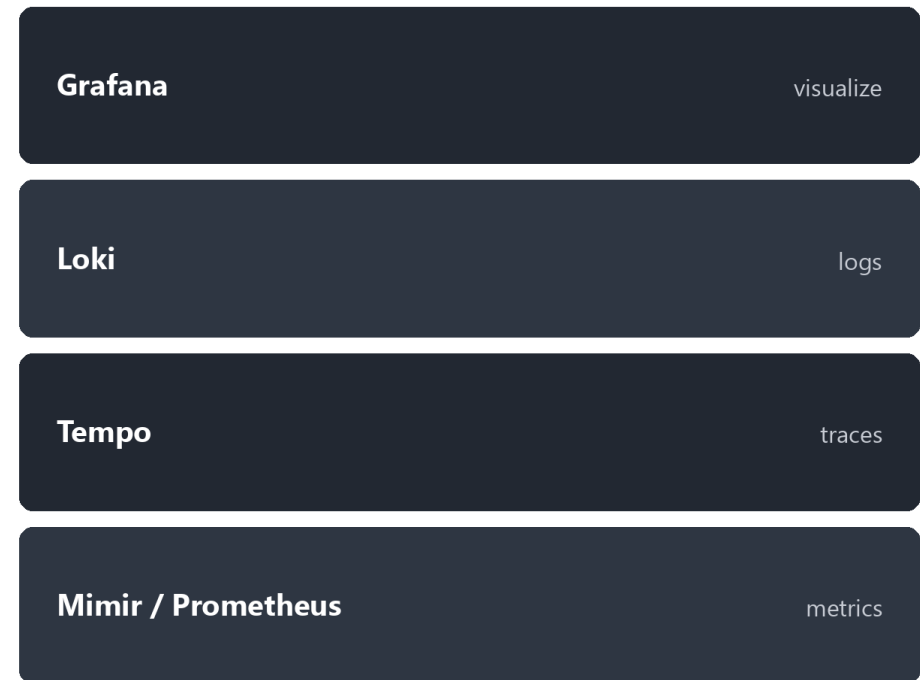


THE LGTM STACK

This environment runs the LGTM stack, where each tool owns one signal. Loki handles logs, Grafana visualizes, Tempo stores traces, and a metrics backend completes the picture, so no single failure mode stays invisible.

- Loki for logs
- Grafana for visualization
- Tempo for traces
- Metrics collection

■ The LGTM stack



LOGS WITH LOKI

In the cluster, applications simply write to standard output and the platform handles the rest. Those logs are collected and pushed into Loki, then queried in Grafana, so developers get centralized logging without changing their code.

- Application stdout
- Centralized storage
- Label based queries
- Incident context

■ Logs with Loki



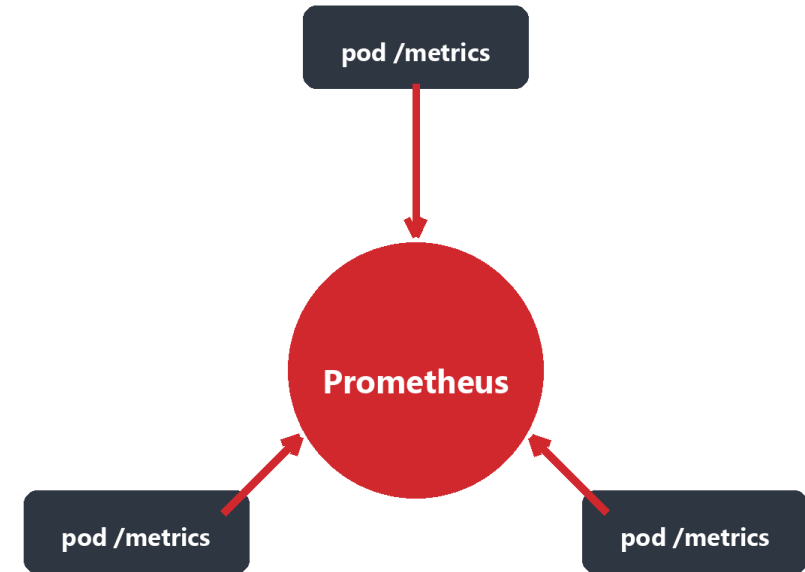
No code changes for developers

METRICS WITH PROMETHEUS

Prometheus collects metrics by scraping endpoints on a schedule, a pull model that fits Kubernetes well. As pods come and go, service discovery keeps the scrape targets current, so new workloads are monitored automatically.

- Pull based collection
- Time series metrics
- Service discovery
- Reliability data

Metrics with Prometheus



Pull model, service discovery keeps targets current

TRACES WITH TEMPO

Tempo stores the distributed traces emitted by applications in the cluster. When a request crosses several services, the trace shows the full path and where the time went, which is essential for debugging microservices.

- Request flows
- Latency breakdown
- Service dependencies
- Distributed debugging

Traces with Tempo

GET /api

service A

service B

db



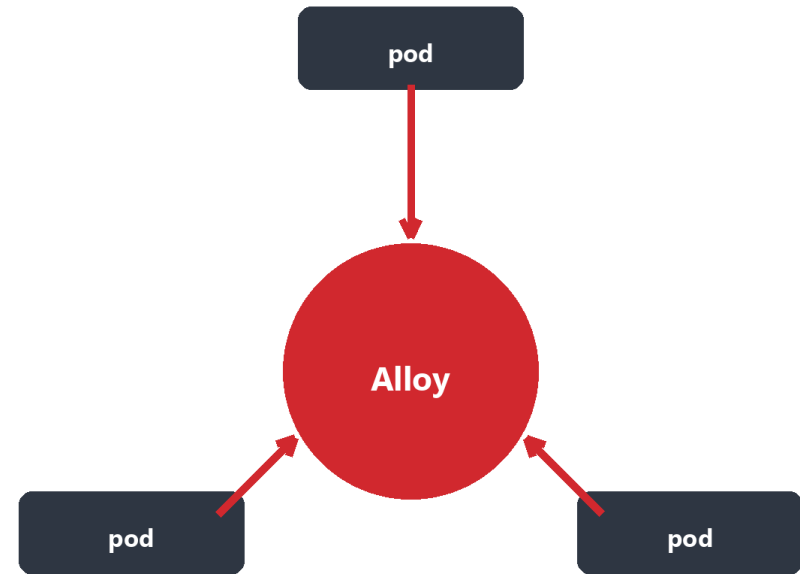
See where the time goes

ALLOY AS THE COLLECTION AGENT

Grafana Alloy runs across the cluster as a single, unified collection agent. It scrapes metrics and forwards telemetry on behalf of your workloads, replacing a pile of separate agents with one component to manage.

- Metrics scraping
- Service monitors
- Unified agent
- Kubernetes native

■ Alloy collection agent



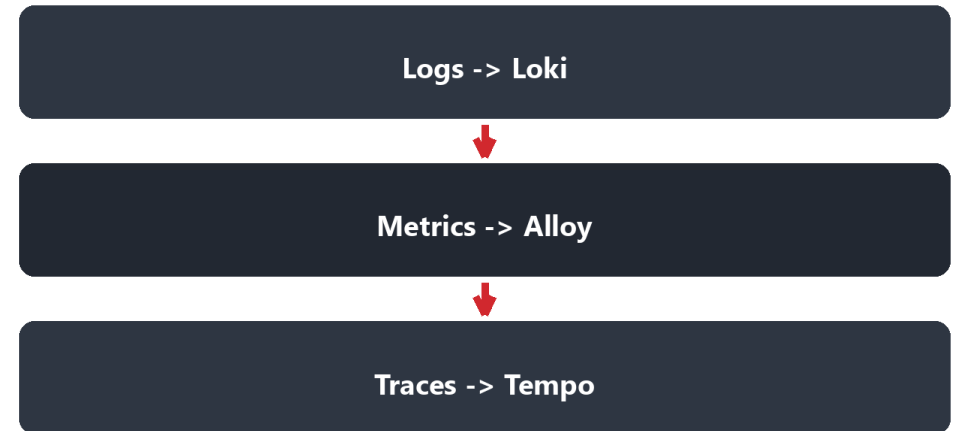
One agent scrapes and forwards telemetry

AUTOMATIC TELEMETRY COLLECTION

On a well-built platform, applications become observable without each team wiring up collection by hand. Logs flow to Loki, metrics through Alloy, and traces to Tempo, so a new service is monitored the day it ships.

- Logs to Loki
- Metrics via Alloy
- Traces to Tempo
- Minimal app changes

■ Automatic collection



A new service is observable on day one

GRAFANA AS THE SINGLE VIEW

All the telemetry in the world is hard to use if it lives in separate tools. Grafana ties metrics, logs, and traces into one interface, so an engineer can pivot from a graph straight to the exact logs without leaving the page.

- Metrics dashboards
- Log exploration
- Trace views
- Correlation



LAB FOCUS

This lab is about living inside a working observability platform, not memorizing query syntax. You will explore a production-style, microservice-ready stack and see how the pieces fit, which is the context that makes the syntax matter later.

- Infrastructure first
- Observability platform
- Microservice ready
- Hands on exploration

What you will explore

- ✓ A working LGTM stack
- ✓ Metrics, logs and traces
- ✓ One Grafana view
- ✓ Microservice ready

MCQ

Why do observability tools thrive in microservice environments?

- Microservices reduce logging needs
- Distributed systems increase complexity
- Metrics replace application logic
- Containers eliminate failures



ANSWER

Why do observability tools thrive in microservice environments?

- Microservices reduce logging needs
- **Distributed systems increase complexity**
- Metrics replace application logic
- Containers eliminate failures



MCQ

Which LGTM component is responsible for storing logs?

- Prometheus
- Tempo
- Loki
- Alloy



ANSWER

Which LGTM component is responsible for storing logs?

- Prometheus
- Tempo
- **Loki**
- Alloy



MCQ

How does Prometheus collect metrics in Kubernetes?

- Applications push metrics to Prometheus
- Prometheus scrapes metrics endpoints
- Metrics are written to logs
- Grafana polls services directly



ANSWER

How does Prometheus collect metrics in Kubernetes?

- Applications push metrics to Prometheus
- **Prometheus scrapes metrics endpoints**
- Metrics are written to logs
- Grafana polls services directly



MCQ

What role does Alloy play in the monitoring stack?

- It visualizes telemetry
- It stores logs
- It collects and forwards telemetry
- It replaces Prometheus



ANSWER

What role does Alloy play in the monitoring stack?

- It visualizes telemetry
- It stores logs
- **It collects and forwards telemetry**
- It replaces Prometheus



MCQ

Why is Grafana described as a single pane of glass?

- It replaces all monitoring tools
- It stores telemetry data
- It unifies metrics, logs, and traces
- It runs inside applications



ANSWER

Why is Grafana described as a single pane of glass?

- It replaces all monitoring tools
- It stores telemetry data
- **It unifies metrics, logs, and traces**
- It runs inside applications



CONGRATULATIONS

Over this module you built a working observability workflow with Grafana, from raw telemetry to centralized logs and dashboards you can actually investigate with.

The skills you practiced here, reading metrics, querying logs with LogQL, and correlating signals during an incident, are the same ones platform and SRE teams use in production every day.

Take this stack, point it at your own services, and keep building the dashboards and alerts your team will rely on.

